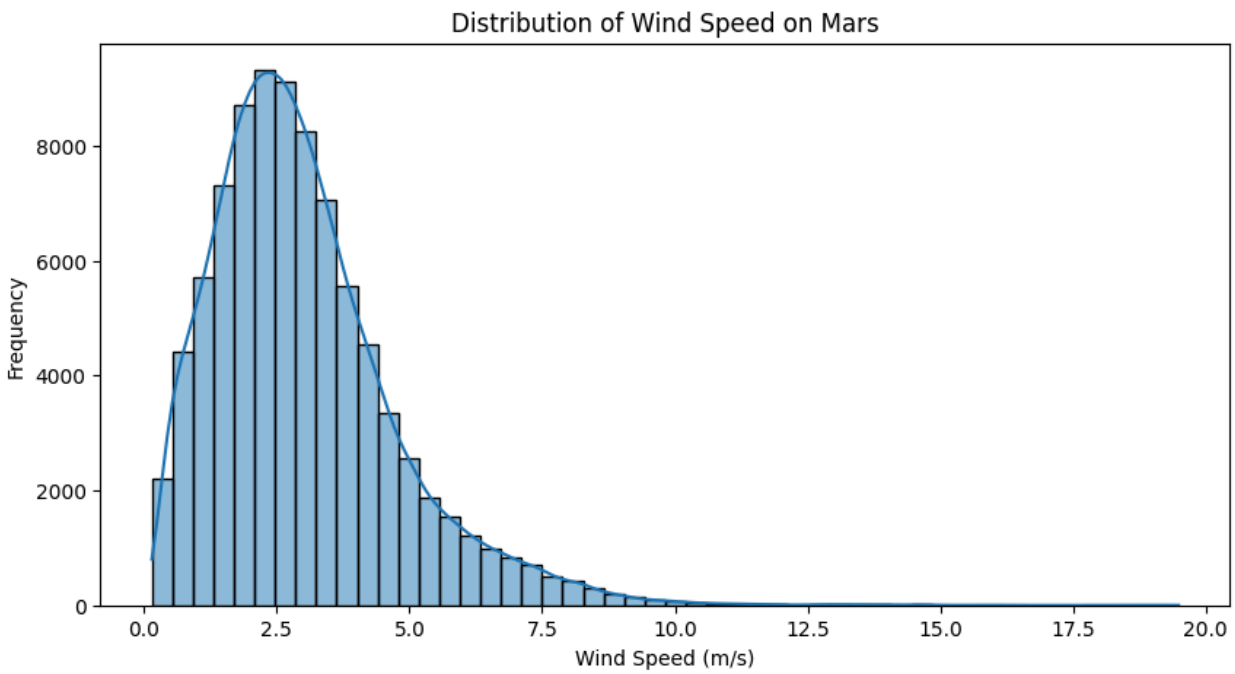
In [1]:
```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load dataset
df = pd.read_csv("mars_wind_downsampled.csv")

# Display basic info
print(df.shape)
print(df.head())
print(df.describe())
```

```
(89378, 6)
   sol LMST_minute  BOOM1_HORIZONTAL_WIND_SPEED  BOOM2_HORIZONTAL_WIND_SPEED  \
0   36       1:32:                     0.404902                     0.795294
1   36       1:33:                     0.301129                     0.497016
2   36       1:34:                     0.560976                     1.012195
3   36       1:35:                     0.962927                     1.395203
4   36       1:36:                     1.875565                     2.044113

   BOOM1_WIND_DIRECTION  BOOM2_WIND_DIRECTION
0            114.981765            149.197451
1            160.529677            177.545806
2            110.529350            132.317967
3            110.570000            121.586260
4            110.570000            111.255484
                sol  BOOM1_HORIZONTAL_WIND_SPEED  BOOM2_HORIZONTAL_WIND_SPEED
\
count  89378.000000                 88433.000000                 88123.000000
mean     179.039596                     2.717584                     3.287207
std       83.464263                     1.791611                     1.985023
min       36.000000                     0.014643                     0.138571
25%      106.000000                     1.422258                     1.896504
50%      180.000000                     2.417984                     2.952358
75%      250.000000                     3.633243                     4.232929
max      315.000000                    32.864677                    31.325000

       BOOM1_WIND_DIRECTION  BOOM2_WIND_DIRECTION
count          88433.000000          88123.000000
mean             172.627692            166.883249
std               70.025418             69.132906
min                3.750000              1.550000
25%              120.860000            111.182349
50%              166.880806            165.424630
75%              222.647097            217.049297
max              353.977903            356.610000
```

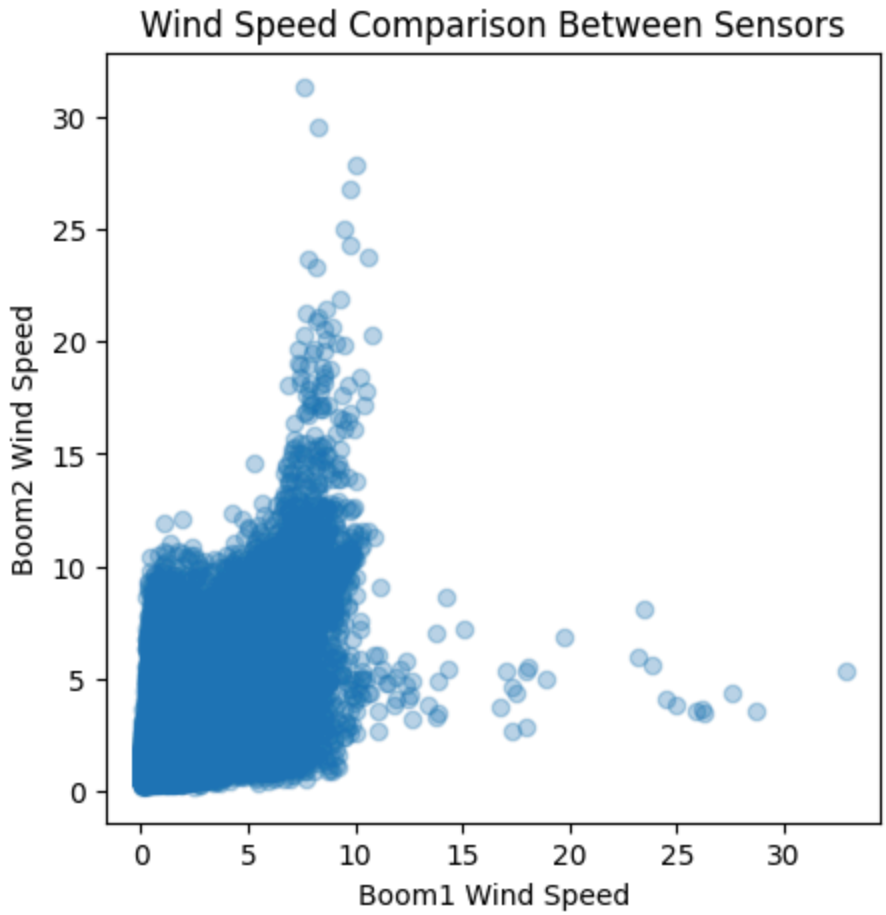
In [2]:
```python
df["AVG_WIND_SPEED"] = (
    df["BOOM1_HORIZONTAL_WIND_SPEED"] +
    df["BOOM2_HORIZONTAL_WIND_SPEED"]
) / 2
```

In [21]:
```python
plt.figure(figsize=(10,5))
```

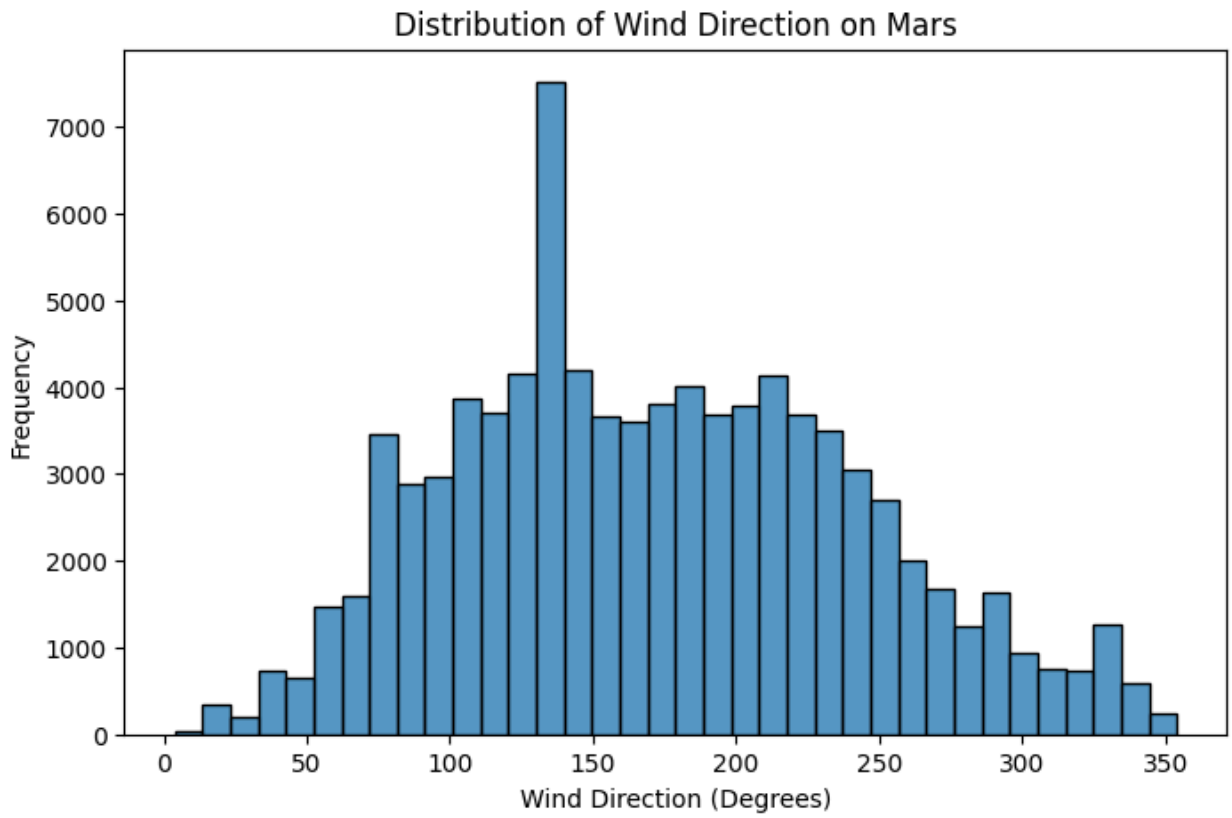
```
sns.histplot(df["AVG_WIND_SPEED"], bins=50, kde=True)

plt.title("Distribution of Wind Speed on Mars")
plt.xlabel("Wind Speed (m/s)")
plt.ylabel("Frequency")

plt.show()
```



In [22]:
```
plt.figure(figsize=(5,5))

plt.scatter(
    df["BOOM1_HORIZONTAL_WIND_SPEED"],
    df["BOOM2_HORIZONTAL_WIND_SPEED"],
    alpha=0.3
)

plt.xlabel("Boom1 Wind Speed")
plt.ylabel("Boom2 Wind Speed")

plt.title("Wind Speed Comparison Between Sensors")

plt.show()
```

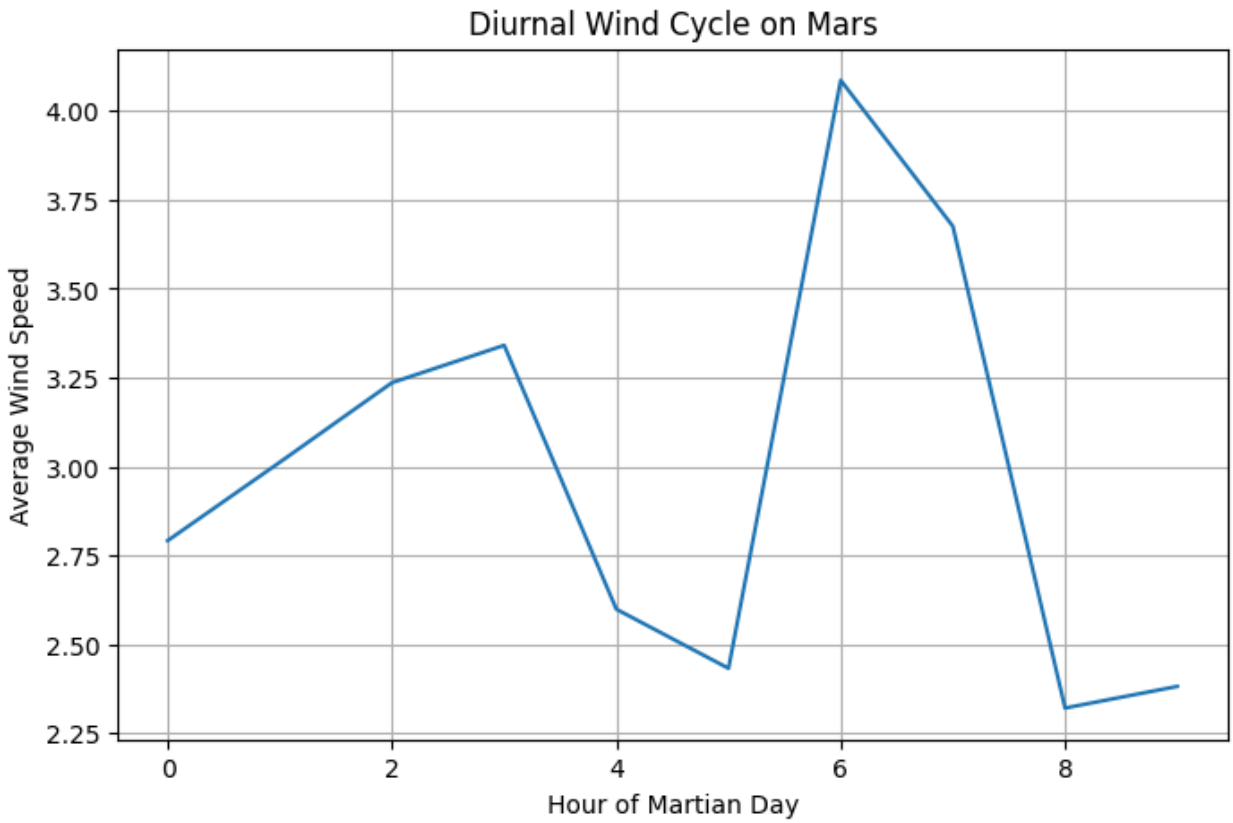


In [23]:
```
plt.figure(figsize=(8,5))

sns.histplot(df["BOOM1_WIND_DIRECTION"], bins=36)

plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Frequency")

plt.title("Distribution of Wind Direction on Mars")

plt.show()
```

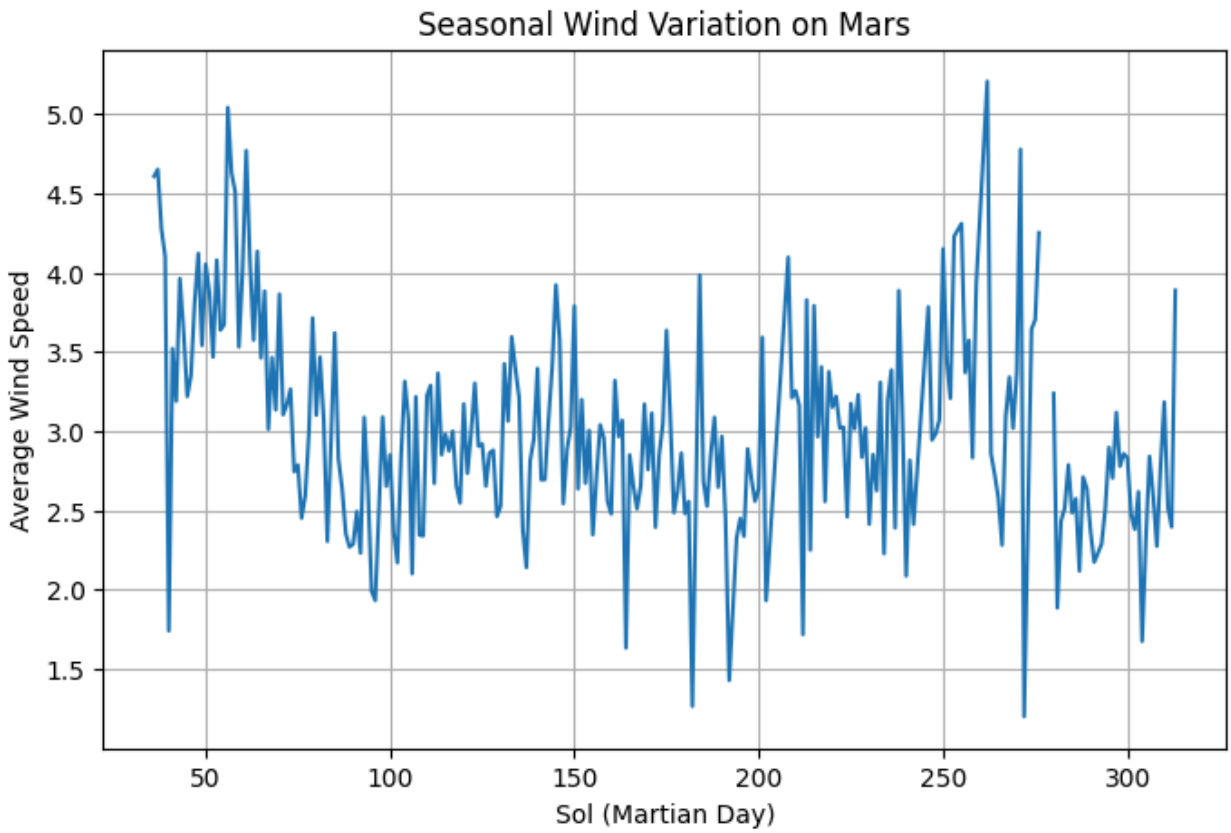


In [24]:
```
# Extract hour
df["HOUR"] = df["LMST_minute"].str.split(":").str[0]
df["HOUR"] = df["HOUR"].astype(int)
```

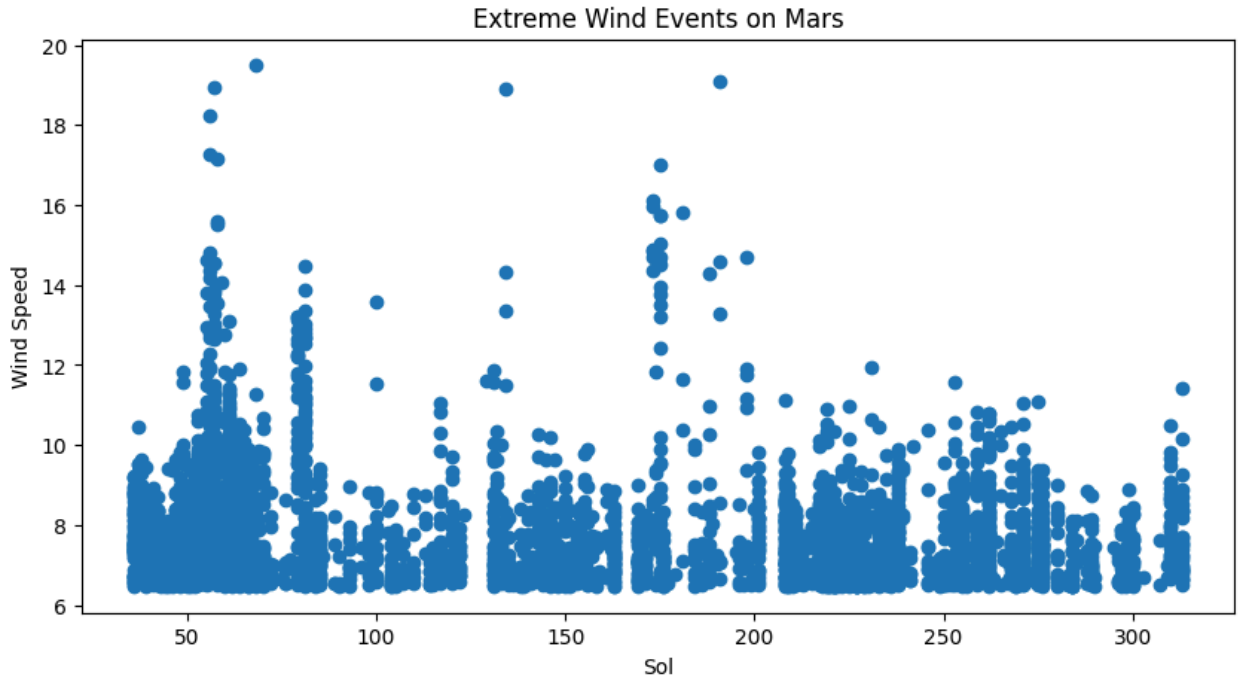
In [25]:
```
diurnal_wind = df.groupby("HOUR")["AVG_WIND_SPEED"].mean()

plt.figure(figsize=(8,5))

plt.plot(diurnal_wind)

plt.xlabel("Hour of Martian Day")
plt.ylabel("Average Wind Speed")

plt.title("Diurnal Wind Cycle on Mars")

plt.grid()

plt.show()
```



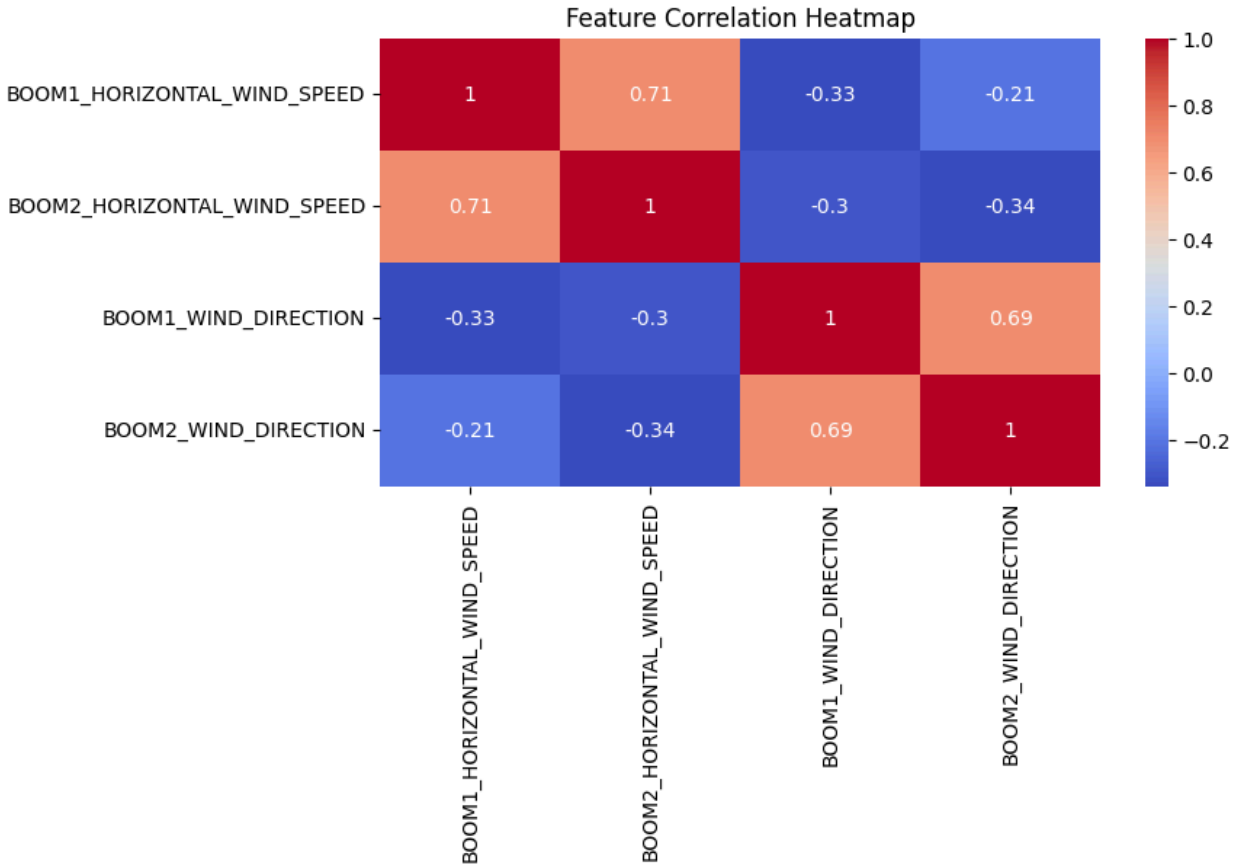
In [26]:
```
seasonal_wind = df.groupby("sol")["AVG_WIND_SPEED"].mean()

plt.figure(figsize=(8,5))

plt.plot(seasonal_wind)

plt.xlabel("Sol (Martian Day)")
plt.ylabel("Average Wind Speed")

plt.title("Seasonal Wind Variation on Mars")

plt.grid()

plt.show()
```



In [27]:
```
threshold = df["AVG_WIND_SPEED"].mean() + 2 * df["AVG_WIND_SPEED"].std()

extreme_wind = df[df["AVG_WIND_SPEED"] > threshold]

plt.figure(figsize=(10,5))

plt.scatter(extreme_wind["sol"], extreme_wind["AVG_WIND_SPEED"])

plt.xlabel("Sol")
plt.ylabel("Wind Speed")

plt.title("Extreme Wind Events on Mars")

plt.show()
```



In [28]:
```
plt.figure(figsize=(8,4))

corr = df[[
"BOOM1_HORIZONTAL_WIND_SPEED",
"BOOM2_HORIZONTAL_WIND_SPEED",
"BOOM1_WIND_DIRECTION",
"BOOM2_WIND_DIRECTION"
]].corr()

sns.heatmap(corr, annot=True, cmap="coolwarm")

plt.title("Feature Correlation Heatmap")

plt.show()
```



In [29]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [30]:
```
df = pd.read_csv("mars_wind_downsampled.csv")

# Average wind speed from both sensors
df["AVG_WIND_SPEED"] = (
    df["BOOM1_HORIZONTAL_WIND_SPEED"] +
    df["BOOM2_HORIZONTAL_WIND_SPEED"]
) / 2

# Average wind direction
df["AVG_DIRECTION"] = (
    df["BOOM1_WIND_DIRECTION"] +
    df["BOOM2_WIND_DIRECTION"]
) / 2

print(df.head())
```

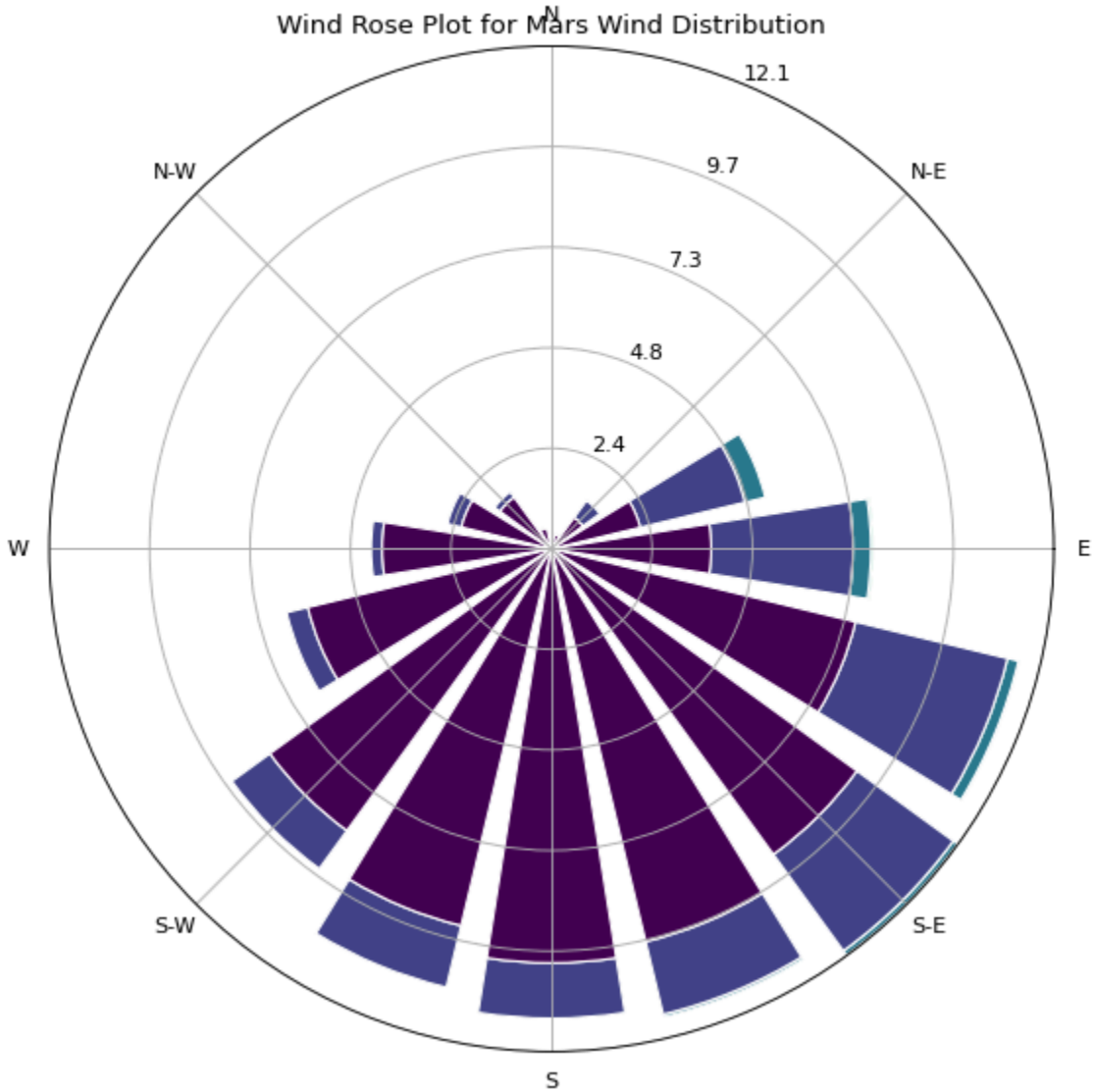
```
   sol LMST_minute  BOOM1_HORIZONTAL_WIND_SPEED  BOOM2_HORIZONTAL_WIND_SPEED  \
0   36       1:32:                     0.404902                     0.795294
1   36       1:33:                     0.301129                     0.497016
2   36       1:34:                     0.560976                     1.012195
3   36       1:35:                     0.962927                     1.395203
4   36       1:36:                     1.875565                     2.044113

   BOOM1_WIND_DIRECTION  BOOM2_WIND_DIRECTION  AVG_WIND_SPEED  AVG_DIRECTION
0            114.981765            149.197451        0.600098     132.089608
1            160.529677            177.545806        0.399073     169.037742
2            110.529350            132.317967        0.786585     121.423659
3            110.570000            121.586260        1.179065     116.078130
4            110.570000            111.255484        1.959839     110.912742
```

In [31]:
```
pip install windrose
```

```
Collecting windrose
  Downloading windrose-1.9.2-py3-none-any.whl.metadata (5.2 kB)
Requirement already satisfied: matplotlib>=3 in /usr/local/lib/python3.12/dist-
packages (from windrose) (3.10.0)
Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3.12/dist-pa
ckages (from windrose) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib>=3->windrose) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-p
ackages (from matplotlib>=3->windrose) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib>=3->windrose) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib>=3->windrose) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dis
t-packages (from matplotlib>=3->windrose) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pack
ages (from matplotlib>=3->windrose) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib>=3->windrose) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.1
2/dist-packages (from matplotlib>=3->windrose) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.7->matplotlib>=3->windrose) (1.17.0)
Downloading windrose-1.9.2-py3-none-any.whl (20 kB)
Installing collected packages: windrose
Successfully installed windrose-1.9.2
```

In [32]:
```
from windrose import WindroseAxes

ax = WindroseAxes.from_ax()

ax.bar(
    df["AVG_DIRECTION"],
    df["AVG_WIND_SPEED"],
    normed=True,
    opening=0.8,
    edgecolor='white'
```

```
)

ax.set_title("Wind Rose Plot for Mars Wind Distribution")

plt.show()
```



This plot shows:

1.dominant wind directions on Mars

2.which directions produce strongest winds

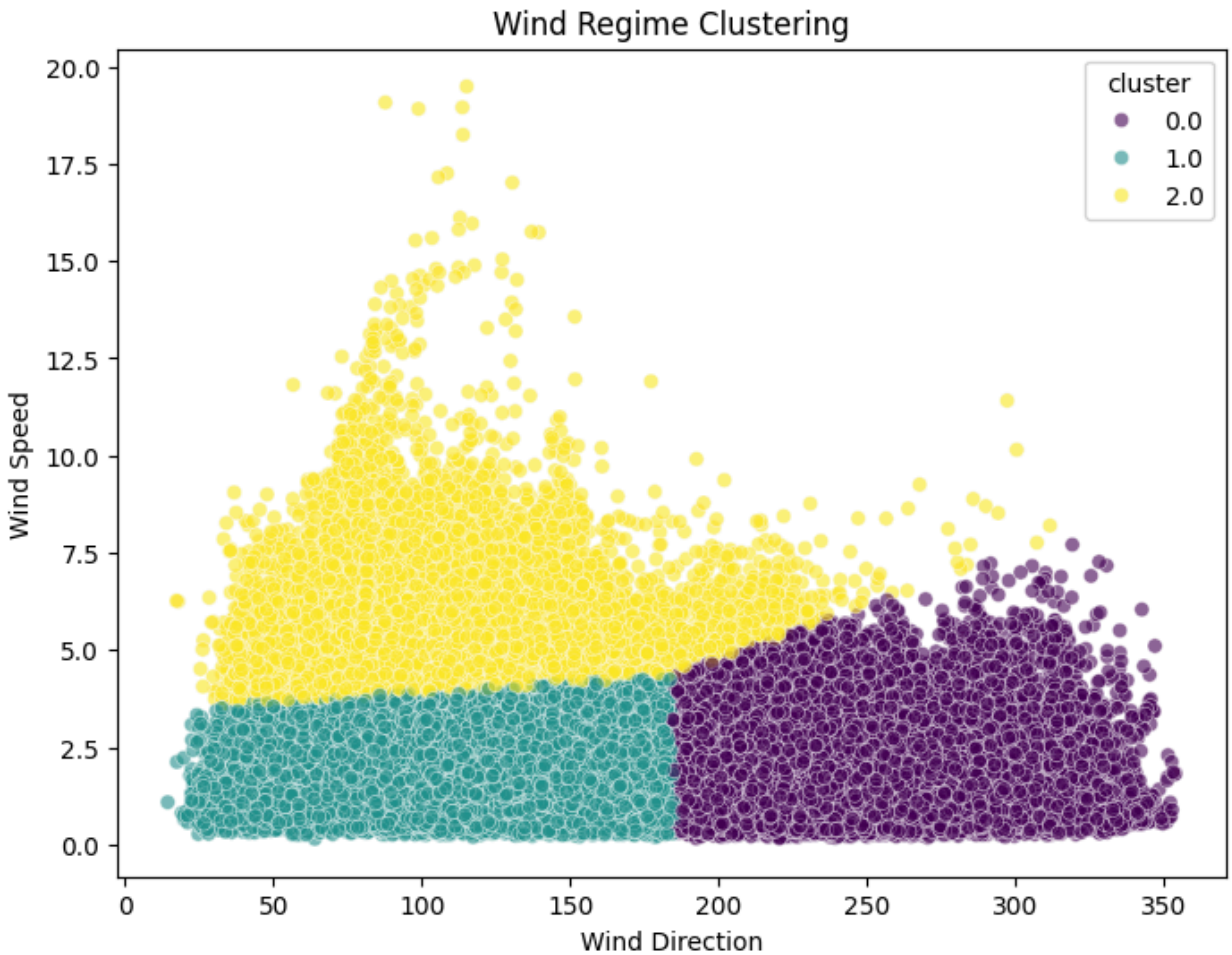
In [33]:
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

Wind Pattern Clustering (K-Means)

Cluster	Meaning
Cluster 0	Calm wind
Cluster 1	Moderate wind
Cluster 2	Strong wind

In [36]:
```
features = df[["AVG_WIND_SPEED","AVG_DIRECTION"]]

features = features.dropna()

scaler = StandardScaler()
X = scaler.fit_transform(features)

kmeans = KMeans(n_clusters=3, random_state=42, n_init='auto')

df["cluster"] = np.nan # Initialize 'cluster' column with NaNs
df.loc[features.index, "cluster"] = kmeans.fit_predict(X)
```

In [39]:
```
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=df["AVG_DIRECTION"],
    y=df["AVG_WIND_SPEED"],
    hue=df["cluster"],
    palette="viridis",
    alpha=0.6
)

plt.xlabel("Wind Direction")
plt.ylabel("Wind Speed")
plt.title("Wind Regime Clustering")

plt.show()
```



LSTM / GRU

In [55]:
```
from sklearn.preprocessing import MinMaxScaler
```

In [56]:
```
data = df["AVG_WIND_SPEED"].dropna().values.reshape(-1,1)

scaler = MinMaxScaler()

data_scaled = scaler.fit_transform(data)
```

In [57]:
```
def create_sequences(data, seq_length):

    X = []
    y = []

    for i in range(len(data)-seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length])

    return np.array(X), np.array(y)

seq_length = 20
```

```
X, y = create_sequences(data_scaled, seq_length)
```

In [58]:
```
split = int(0.8 * len(X))

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]
```

In [59]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

In [60]:
```
lstm_model = Sequential()

lstm_model.add(LSTM(50, return_sequences=True, input_shape=(seq_length,1)))
lstm_model.add(LSTM(50))
lstm_model.add(Dense(1))

lstm_model.compile(
    optimizer="adam",
    loss="mse"
)

lstm_model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64
)
```

```
Epoch 1/10
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(**kwargs)
```

```
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 28s 23ms/step - loss: 0.0034
Epoch 2/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 25s 23ms/step - loss: 0.0022
Epoch 3/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 41s 23ms/step - loss: 0.0022
Epoch 4/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 25s 23ms/step - loss: 0.0022
Epoch 5/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 25s 23ms/step - loss: 0.0022
Epoch 6/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 41s 23ms/step - loss: 0.0022
Epoch 7/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 41s 23ms/step - loss: 0.0021
Epoch 8/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 25s 23ms/step - loss: 0.0022
Epoch 9/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 25s 23ms/step - loss: 0.0022
Epoch 10/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 26s 24ms/step - loss: 0.0021
```

Out[60]:

```
<keras.src.callbacks.history.History at 0x7a43a450c5f0>
```

In [62]:

```
from tensorflow.keras.layers import GRU
```

In [63]:

```
gru_model = Sequential()

gru_model.add(GRU(50, return_sequences=True, input_shape=(seq_length,1)))
gru_model.add(GRU(50))
gru_model.add(Dense(1))

gru_model.compile(
    optimizer="adam",
    loss="mse"
)

gru_model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64
)
```

```
Epoch 1/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 35s 28ms/step - loss: 0.0028
Epoch 2/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 30s 27ms/step - loss: 0.0022
Epoch 3/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 42s 28ms/step - loss: 0.0021
Epoch 4/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 41s 29ms/step - loss: 0.0021
Epoch 5/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 40s 28ms/step - loss: 0.0021
Epoch 6/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 30s 27ms/step - loss: 0.0022
Epoch 7/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 30s 27ms/step - loss: 0.0022
Epoch 8/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 42s 29ms/step - loss: 0.0022
Epoch 9/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 29s 27ms/step - loss: 0.0022
Epoch 10/10
1090/1090 ━━━━━━━━━━━━━━━━━━━━ 30s 28ms/step - loss: 0.0021
```

Out[63]:
```
<keras.src.callbacks.history.History at 0x7a43b110b0e0>
```

In [64]:
```
lstm_pred = lstm_model.predict(X_test)
gru_pred = gru_model.predict(X_test)

lstm_pred = scaler.inverse_transform(lstm_pred)
gru_pred = scaler.inverse_transform(gru_pred)

y_test_actual = scaler.inverse_transform(y_test)
```

```
545/545 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step
545/545 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step
```

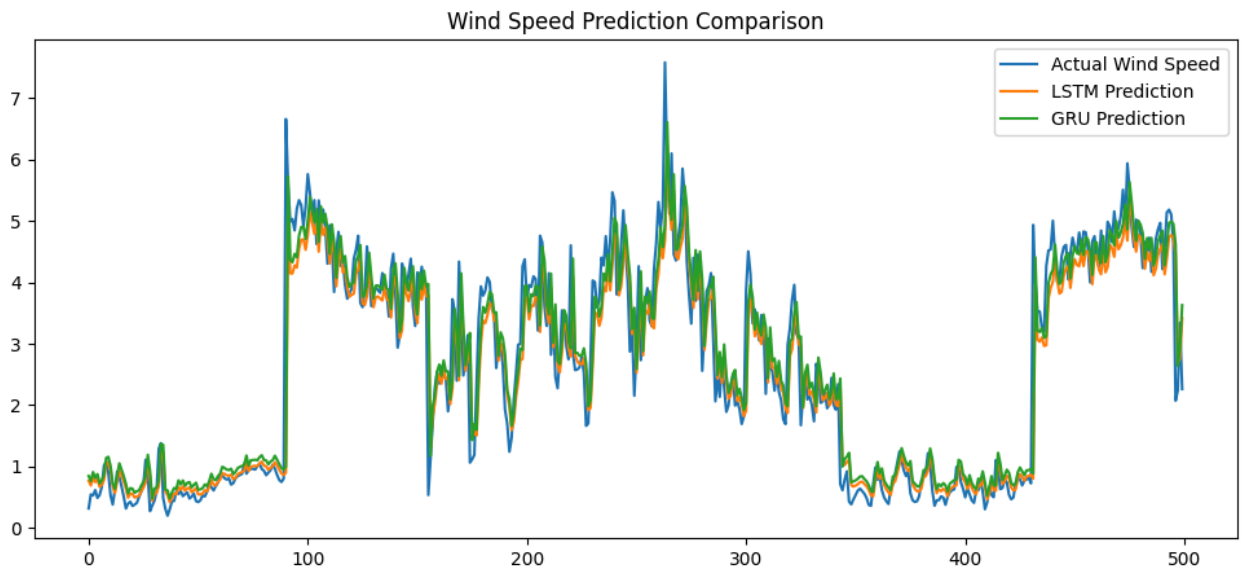
In [49]:
```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

In [67]:
```
def evaluate_model(y_true, y_pred):

    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    return mae, rmse, r2
```

In [75]:
```
lstm_metrics = evaluate_model(y_test_actual, lstm_pred)
gru_metrics = evaluate_model(y_test_actual, gru_pred)

print("LSTM:", lstm_metrics)
print("GRU:", gru_metrics)
```

```
LSTM: (0.49012556071380436, np.float64(0.7338148906488362), 0.7667386359655328)
GRU: (0.5038115181168537, np.float64(0.734073985897731), 0.7665738871127362)
```

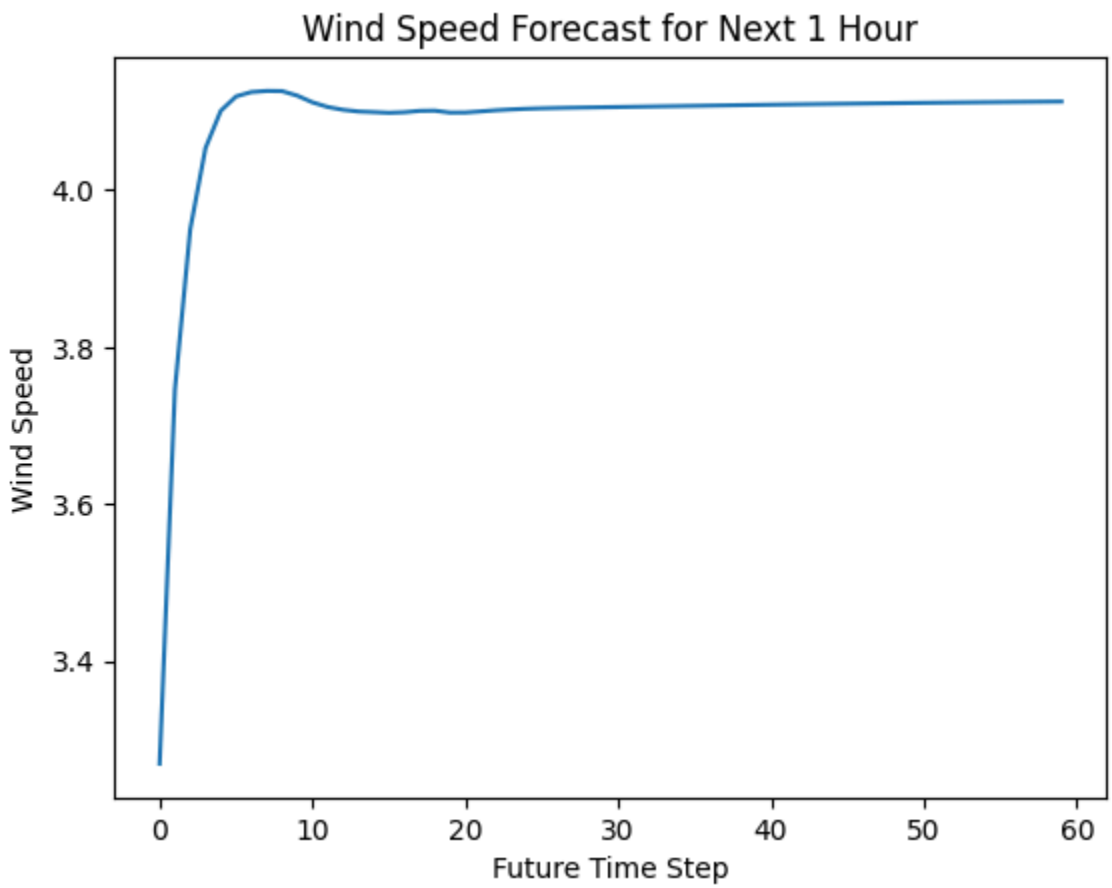
In [76]:
```
plt.figure(figsize=(12,5))

plt.plot(y_test_actual[:500], label="Actual Wind Speed")
plt.plot(lstm_pred[:500], label="LSTM Prediction")
plt.plot(gru_pred[:500], label="GRU Prediction")

plt.legend()

plt.title("Wind Speed Prediction Comparison")

plt.show()
```



In [78]:
```
last_sequence = data_scaled[-seq_length:]

predictions = []

current_seq = last_sequence.reshape(1,seq_length,1)

for i in range(60):

    pred = gru_model.predict(current_seq)

    predictions.append(pred[0,0])

    # Reshape pred to (1, 1, 1) to match dimensions of current_seq
    current_seq = np.append(current_seq[:,1:,:], pred.reshape(1,1,1), axis=1)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
```

In [79]:
```
predictions = scaler.inverse_transform(np.array(predictions).reshape(-1,1))
```

In [80]:
```
plt.plot(predictions)

plt.title("Wind Speed Forecast for Next 1 Hour")

plt.xlabel("Future Time Step")
plt.ylabel("Wind Speed")

plt.show()
```